- Edy Lane Martínez Rojas 202510939

- Isabella Casallas Abril - 202510253

Proyecto 2 – Correcciones a la gramática de Objectilang respecto al Proyecto 1

Elementos Esenciales de Lenguajes de Programación

Solución del Proyecto 1 utilizada: la de nuestro propio grupo. No se usó la solución de ningún otro grupo.

Este documento explica todos los cambios que se hicieron a la gramática EBNF del Proyecto 1 al implementar Objectilang en Rascal (archivo Syntax.rsc), y la razón de cada uno más a fondo, a parte del documento de la entrega 1 corregida. Los cambios también aparecen resaltados en el documento del Proyecto 1, que se entrega editado junto con este:D.

Los cambios son de dos tipos. Las correcciones arreglan errores o ambigüedades de la gramática original: sin ellas el lenguaje no funcionaría bien, porque Rascal rechazaría programas válidos o encontraría más de una forma de leer el mismo programa (una ambigüedad), y en ese caso no puede construir un único árbol. Las adaptaciones cambian la forma de escribir la gramática sin cambiar los programas que se aceptan; son necesarias porque Rascal tiene su propia notación y porque el AST se construye automáticamente a partir de la gramática. Estos mismos cambios aparecen en el documento auxiliar entregado de forma mas especifica.

PARTE 1. CORRECCIONES (ERRORES DE LA GRAMÁTICA DEL PROYECTO 1)

Corrección 1. Identificadores y palabras reservadas

Problema.

En el Proyecto 1 la regla Identificador ::= Letra (Letra | Num)* generaba cualquier palabra, incluidas las reservadas. En una nota se decía que una palabra reservada no podía ser identificador, pero también se aclaraba que esa restricción no se podía expresar en EBNF. Esto deja dos problemas. Primero, object o if podrían usarse como nombres (por ejemplo val if = 3). Segundo, aunque se prohibiera la palabra exacta, un identificador que empieza con una palabra reservada seguía siendo ambiguo: negative + 1 se podía leer como el identificador negative o como el operador neg aplicado al identificador ative. Lo mismo pasaba con valores (val + ores), newton (new + ton) o iffy (if + fy). Como en la gramática no hay ningún espacio obligatorio entre una palabra clave y lo que sigue, las dos lecturas eran válidas.

Corrección. En Rascal la restricción sí se puede expresar. Las palabras reservadas se agrupan en un no terminal keyword y se restan del conjunto de identificadores con el operador \

(resta). Además, se agrega la restricción de seguimiento !>> [a-zA-Z0-9], que significa "no puede venir seguido de una letra o un dígito":

```
lexical Id = @category="variable" ([a-zA-Z][a-zA-Z0-9]* !>> [a-zA-Z0-9]) \ Reservada;

keyword Reservada
  = "object" | "extends" | "method" | "val" | "var" | "new"
  | "override" | "if" | "then" | "else" | "for" | "in"
  | "String" | "Integer" | "Boolean" | "true" | "false"
  | "and" | "or" | "neg"
  ;
```

La misma restricción se aplica a cada palabra clave dentro de las reglas, por ejemplo:

```
syntax Modificador = inmutable: "val" !>> [a-zA-Z0-9] | mutable: "var" !>> [a-zA-Z0-9];
negacion: "neg" !>> [a-zA-Z0-9] Operacion
```

Por qué esta solución. La resta \ Reservada es exactamente la nota del Proyecto 1, ahora escrita de manera formal. La restricción en Id obliga a que un identificador tome la palabra completa: termina solo cuando lo que sigue ya no es letra ni dígito, así que perro nunca se parte en per + ro. La restricción en las palabras clave impide que neg se separe de ative: después de neg no puede venir una letra. Juntas, las dos restricciones implementan la regla de la cadena más larga que usan casi todos los lenguajes de programación. Con esto, negative, valores y newton son siempre identificadores, y objectDog es un identificador y no object Dog.

Corrección 2. and, or y neg como palabras reservadas

Problema. En el Proyecto 1, and, or y neg solo aparecían en el conjunto de operadores, no en el de palabras reservadas.

Corrección. Se agregaron a Reservada (ver Corrección 1).

Por qué. A diferencia de + o *, estos operadores se escriben con letras, así que también cumplen la regla de Identificador. Si no son reservados, x and y se podría leer como la conjunción de x y y, o como tres identificadores seguidos, y and podría usarse como nombre de variable.

Corrección 3. if y for dentro de las expresiones

Problema. En el Proyecto 1, CondicionalIf y CicloFor eran alternativas de ElementoBase, es decir, estaban al nivel más bajo de las expresiones, el mismo de un número o un identificador:

```
Primario ::= ElementoBase Argumentos? Sufijo*
ElementoBase ::= ... | CondicionalIf | CicloFor | ...
CondicionalIf ::= "if" Expresion "then" Sentencia ("else" Sentencia)?
CicloFor ::= "for" "(" VariableIteracion "<-" Expresion ")" Sentencia
```

Como la última parte de un if o un for es una Sentencia, que puede ser cualquier expresión, la gramática no indica dónde termina el if. Esto produce varias ambigüedades:

- if a then b else c + 1 se puede leer como (if a then b else c) + 1 o como if a then b else (c + 1).

- if a then b else c.m se puede leer como (if a then b else c).m o como if a then b else (c.m).
- for (i <- xs) f(i) + 1 se puede leer como (for ...) + 1 o como for (...) (f(i) + 1).
- if a then if b then c else d tiene el problema conocido como else colgante: el else puede pertenecer al if interno o al externo.

Primer intento y por qué no funcionó. Primero se sacaron if y for de ElementoBase y se pusieron como el nivel de menor prioridad de la regla de expresiones, debajo de las flechas. Eso resolvió los casos anteriores, pero al probar el programa if a then x + if b then c else d, Rascal encontró una ambigüedad nueva: además de if a then (x + if b then c else d), aceptaba la lectura (if a then x) + (if b then c else d), es decir, una suma de dos ifs. Esto pasa porque un if sin else no tiene un símbolo que lo cierre, entonces la prioridad no basta para saber hasta dónde llega cuando aparece en medio de una operación.

Corrección final (Correción mas difícil del trabajo). Las expresiones se dividen en dos niveles. Expresion contiene el if, el for y las operaciones. Operacion contiene todos los operadores, con la misma precedencia del Proyecto 1 (ver Adaptación 3):

```
syntax Expresion
  = ifSinElse: "if" !>> [a-zA-Z0-9] Expresion "then" Expresion () !>> "else"
  | ifConElse: "if" !>> [a-zA-Z0-9] Expresion "then" Expresion "else" Expresion
  | ciclo: "for" !>> [a-zA-Z0-9] "(" VariableIteracion "\<-" Expresion ")" Expresion
  | operacion: Operacion
  ;

syntax Operacion
  = base: Primario
  | bloqueExp: Bloque
  > negacion: "neg" !>> [a-zA-Z0-9] Operacion | negativo: "-" Operacion
  > right potencia: Operacion "**" Operacion
  > ...
  ;
```

Por qué esta solución. Los operandos de un operador son de tipo Operacion, no Expresion. Por eso un if o un for nunca pueden quedar sueltos dentro de una suma, una comparación o cualquier otra operación, y las lecturas dobles desaparecen de raíz, no solo para la suma sino para todos los operadores (por ejemplo, también en a ** if b then c + d o en neg if a then x + y). Si alguien necesita un if dentro de una operación, lo escribe entre paréntesis, porque la agrupación "(" Expresion ")" sí acepta cualquier Expresion: x + (if b then c else d). Es la misma convención de muchos lenguajes, donde una expresión condicional usada como operando va entre paréntesis.

El else colgante. Se resuelve con () !>> "else" al final de ifSinElse. El símbolo () marca una posición (después de los espacios que siguen a la rama then), y !>> "else" dice que en esa posición no puede empezar la palabra else. Es decir: un if sin else no puede estar seguido de un else. En if a then if b then c else d, la lectura en la que el else pertenece al if externo exige que el if interno (if b then c) no tenga else y esté seguido de else, y eso queda prohibido. Solo queda la lectura en la que el else pertenece al if más cercano, que es la convención de casi todos los lenguajes. Se eligió esta forma, y no una exclusión directa como

Expresion!ifSinElse, porque la restricción funciona sin importar qué tan adentro esté el if interno.

Las ramas pasan de Sentencia a Expresion. En el Proyecto 1 las ramas del if y el cuerpo del for eran Sentencia. Ahora son Expresion, porque if y for pasaron a ser expresiones y una expresión debe estar formada por expresiones. Así un if también se puede usar donde se espera un valor, por ejemplo val x = if a then 1 else 2. Además, con Sentencia las ramas podrían ser declaraciones, que no producen un valor. Para que un bloque siga pudiendo ser rama (if c then { ... } else { ... }), Bloque se agregó como alternativa de Operacion (bloqueExp).

Consecuencias. Un if o un for dentro de una operación necesita paréntesis: (if a then b else c) + 1 y 1 + (if a then b else c). Sin paréntesis, if a then b else c + 1 se lee siempre como if a then b else (c + 1), y 1 + if a then b else c es un error de sintaxis. Una declaración no puede ser rama directa de un if (if c then val x = 1 es un error); hay que ponerla en un bloque: if c then { val x = 1 }. Los no terminales CondicionalIf y CicloFor desaparecen: sus reglas ahora son las alternativas ifSinElse, ifConElse y ciclo de Expresion.

Corrección 4. Sentencia ya no incluye Bloque

Problema. En el Proyecto 1, Sentencia ::= DeclaracionVariable | Expresion | Bloque.

Corrección. Se eliminó la alternativa Bloque de Sentencia:

```
syntax Sentencia
  = sentDeclaracion: DeclaracionVariable
  | sentExpresion: Expresion
  ;
```

Por qué. Desde la Corrección 3, un bloque ya es una expresión (bloqueExp). Si además fuera una sentencia directa, un bloque { ... } dentro de otro bloque se podría derivar de dos formas (como Sentencia -> Bloque o como Sentencia -> Expresion -> Operacion -> Bloque), lo que es ambiguo. Los bloques anidados se siguen pudiendo escribir igual que antes.

Corrección 5. Separación de las sentencias dentro de un bloque

Problema. En el Proyecto 1, Bloque ::= "{" Sentencia* "}" pone las sentencias una tras otra, sin nada entre ellas. Como los espacios y los saltos de línea no tienen significado en la gramática, no siempre se sabe dónde termina una sentencia y empieza la siguiente:

- x -1 puede ser una resta (x - 1) o dos sentencias (x y luego -1).
- print(1) print(2) puede ser dos llamadas o cuatro sentencias: print, (1), print, (2).
- new Foo (1) puede ser new Foo(1) o new Foo seguido de la agrupación (1).

Corrección. Se mantiene Sentencia* sin separador, pero se prohíbe que una sentencia termine justo antes de un (o un -:

```
syntax Bloque = bloque: "{" (Sentencia () !>> "(" !>> "-")* "}";
```

Por qué esta solución. Las dos únicas formas de que la sentencia siguiente "continúe" la anterior son que empiece con ((que se confunde con una llamada o con argumentos) o con -

(que se confunde con una resta). Al prohibir que una sentencia vaya seguida de esos símbolos, el parser siempre toma la lectura en la que el (o el - continúan la expresión anterior. El () se pone después de los espacios, para que la restricción mire el siguiente símbolo real y no un espacio. Se consideró exigir ; entre sentencias, que también elimina la ambigüedad, pero se descartó porque obligaba a cambiar la sintaxis del lenguaje y todos los programas existentes, incluida la Derivación 3 del Proyecto 1. Con la restricción, esos programas siguen siendo válidos tal cual.

Consecuencia. Una sentencia que empiece con (o con - no puede ir justo después de otra: { x -1 } se lee como la resta x - 1, y { f(1) (2) } es un error de sintaxis.

Corrección 6. Argumentos en Primario

Problema. En el Proyecto 1, Primario ::= ElementoBase Argumentos? Sufijo*, y además NuevaInstancia ::= "new" Tipo Argumentos? CuerpoInstancia?. Por eso new Cat(1) tenía dos derivaciones: los argumentos (1) podían ser de la NuevaInstancia o del Primario. Además, la regla permitía poner argumentos después de cualquier elemento, como 1(2) o true(x), que no tienen sentido.

Corrección. Los argumentos se quitan de Primario y solo se permiten pegados a un identificador (una llamada), a un método después de un punto, o a un new:

```
syntax Primario = primario: ElementoBase Sufijo*;

syntax ElementoBase
  = variable: Id
  | llamada: Id Argumentos
  | literal: Literal
  | instancia: NuevaInstancia
  | tupla: Tupla
  | agrupacion: "(" Expresion ")"
  ;

syntax Sufijo = acceso: "." Id | llamadaMetodo: "." Id Argumentos;
```

Por qué. Así cada paréntesis de argumentos tiene un único dueño y la ambigüedad desaparece. También se separan en el AST los dos tipos de sufijo: acceso a un campo (c.nombre) y llamada a un método (c.talk()). Se siguen aceptando print(x), obj.campo, obj.metodo(x) y new Foo(1).bar(2).baz. Ya no se acepta aplicar argumentos a algo que no sea un nombre, como 1(2) o (f)(x).

Corrección 7. Doble opcional en OverrideMetodo

Problema. En el Proyecto 1, OverrideMetodo ::= "override" Identificador "(" Parametros? ")" ":" Tipo "=" Bloque. Pero Parametros ya era opcional por dentro (su regla completa está entre paréntesis con ?), así que los paréntesis vacíos () se podían derivar de dos formas: sin Parametros, o con un Parametros vacío.

Corrección. Se quitó el ?:

```
syntax OverrideMetodo
  = sobrescribir: "override" !>> [a-zA-Z0-9] Id "(" Parametros ")" ":" Tipo "=" Bloque;
```

Por qué. Dos derivaciones para el mismo texto son una ambigüedad. Como Parametros ya acepta la lista vacía, el ? sobraba. Los programas aceptados son los mismos.

Corrección 8. Tipo

Problema. En el Proyecto 1, Tipo ::= ("String" | "Integer" | "Boolean" | Identificador) ("[" Tipo ("," Tipo)* "]")?. Esto permitía tipos sin sentido como String[Integer] o Boolean[A], porque los argumentos genéricos se podían poner después de cualquier tipo.

Corrección. Cada forma del tipo es una alternativa propia, y solo un tipo con nombre puede llevar argumentos genéricos:

```
syntax Tipo
  = tipoString: "String" !>> [a-zA-Z0-9]
  | tipoInteger: "Integer" !>> [a-zA-Z0-9]
  | tipoBoolean: "Boolean" !>> [a-zA-Z0-9]
  | tipoNombre: Id nombre
  | tipoGenerico: Id nombre "[" {Tipo ","}+ argumentos "]"
  ;
```

Por qué. Solo los objetos definidos por el usuario (como List o Map) se declaran con parámetros genéricos, así que solo ellos pueden recibir argumentos genéricos. Además, al separar las alternativas, cada forma del tipo tiene su propio constructor en el AST. La versión original con grupos anónimos tampoco funcionaba en Rascal: List[String] daba error de sintaxis. Se siguen aceptando tipos anidados como Map[K, List[V]].

Corrección 9. Strings e interpolación

Problema 1: se perdían los espacios. En Rascal, en una regla syntax se ignoran automáticamente los espacios entre cada símbolo. Si LiteralString se escribe como regla syntax, como en el Proyecto 1, los espacios de " hola mundo " se descartan, porque Rascal los trata como separadores y no como parte del texto. Además, un espacio al inicio del string se podía leer como texto o como separador, lo que es ambiguo.

Problema 2: muy pocos caracteres permitidos. En el Proyecto 1, CaracterString ::= Letra | Num | " " | "." | "," | ":" | "!". Strings tan comunes como "¿Cómo estás?", "año", "a-b" o "100%" eran inválidos.

Corrección.

```
lexical LiteralString = @category="string" cadena: "\"" ParteString* "\"";

lexical ParteString
  = texto: TextoString
  | interpVariable: "$" Id
  | interpExpresion: "${" Layout Expresion Layout "}";

lexical TextoString = CaracterString+ !>> ![\"$\\\n] !>> [\\];

lexical CaracterString = ![\"$\\\n] | "\\" [\"$\\n];
```

Por qué esta solución. LiteralString y sus partes son lexical, y en una regla lexical Rascal no ignora nada: cada espacio es parte del texto. Dentro de ${ ... }, en cambio, sí se quiere poder

escribir ${ edad + 1 } con espacios, así que ahí se permiten de forma explícita con Layout. CaracterString acepta ahora cualquier carácter excepto ", $, \ y el salto de línea (eso significa ![...]: "cualquier carácter menos estos"). Esos cuatro se excluyen porque tienen un significado especial: " cierra el string, $ empieza una interpolación, \ empieza un escape, y un salto de línea dentro de un string casi siempre es un error. Para escribirlos dentro del texto se usan secuencias de escape: \", \$, \\ y \n. La restricción !>> en TextoString hace que el texto tome todos los caracteres seguidos de una vez, para que "Hola" no se pueda partir en "Ho" + "la" (otra ambigüedad). La interpolación del Proyecto 1 se conserva igual: $nombre y ${expresion}. Interpolacion dejó de ser un no terminal aparte y quedó como dos alternativas de ParteString.

Corrección 10. Inconsistencias del documento del Proyecto 1

- La regla se llamaba VariableIteración (con tilde), pero en el conjunto de no terminales aparecía como VariableIteracion. Se usa el nombre sin tilde en todo el documento.
- La palabra "main" se usa en la regla Metodo pero no aparecía en el alfabeto. Se agregó. No es palabra reservada, porque solo aparece justo después de method y no genera conflictos.
- Se agregaron al alfabeto los símbolos nuevos: ${ (abre una interpolación), \ (escape dentro de strings) y // (comentarios).

PARTE 2. ADAPTACIONES A RASCAL (NO CAMBIAN LOS PROGRAMAS ACEPTADOS, SOLO ES CUESTIÓN DE COMO RASCAL LOS TOMABA)

Adaptación 1. Espacios y comentarios

Cambio. El Proyecto 1 no decía nada sobre espacios en blanco, saltos de línea o comentarios. En Rascal se definen con un no terminal layout, y se agregó el comentario de línea //:

```
layout Layout = EspacioOComentario* !>> [\ \t\n\r] !>> "//";

lexical EspacioOComentario
  = [\ \t\n\r]
  | @category="Comment" "//" ![\n]* $
  ;
```

Por qué. Rascal necesita saber qué ignorar entre los símbolos de una regla; sin layout no se podría escribir ni un espacio. La restricción !>> obliga a que el layout tome todos los espacios y comentarios seguidos. Sin ella, tres espacios se podrían repartir de varias formas (uno aquí y dos allá, etc.), y cada forma sería una lectura distinta del mismo programa. Los comentarios no estaban en el Proyecto 1, pero son útiles para documentar los programas y no cambian ningún programa válido.

Adaptación 2. Reglas léxicas y sintácticas

Cambio. Rascal distingue entre reglas lexical (cómo se forma una palabra: Id, Entero, LiteralString) y reglas syntax (cómo se forman frases con esas palabras). En EBNF todas eran iguales. Letra y Num ya no son reglas aparte: quedan como las clases de caracteres [a-zA-Z] y [0-9], y LiteralEntero quedó como el léxico Entero:

```
lexical Entero = @category="number" [0-9]+ !>> [0-9];
```

Por qué. En las reglas syntax Rascal permite espacios entre cada símbolo; en las lexical, no. Si Id fuera syntax, p e r r o sería un identificador válido. Por eso los identificadores, los números y los strings tienen que ser léxicos. La restricción en Entero cumple la misma función que en Id: tomar el número completo, para que 123 no se lea como 12 y 3.

Adaptación 3. La cascada de expresiones pasa a prioridades

Cambio. En el Proyecto 1 la precedencia de los operadores se expresaba con una cadena de nueve no terminales: ExpresionFlecha -> ExpresionOr -> ExpresionAnd -> ExpresionComparacion -> ExpresionSuma -> ExpresionMult -> ExpresionPotencia -> ExpresionUnaria -> Primario. En Rascal todos los operadores quedan en el no terminal Operacion. Los niveles se separan con > (de mayor a menor prioridad) y cada nivel indica su asociatividad (left, right o non-assoc):

```
syntax Operacion
  = base: Primario
  | bloqueExp: Bloque
  > negacion: "neg" !>> [a-zA-Z0-9] Operacion | negativo: "-" Operacion
  > right potencia: Operacion "**" Operacion
  > left ( mult: Operacion "*" Operacion | div: Operacion "/" Operacion
         | modulo: Operacion "%" Operacion )
  > left ( suma: Operacion "+" Operacion | resta: Operacion "-" Operacion )
  > non-assoc ( menor: Operacion "\<" Operacion | mayor: Operacion "\>" Operacion
              | menorIgual: Operacion "\<=" Operacion | mayorIgual: Operacion "\>=" Operacion
              | distinto: Operacion "\<\>" Operacion | igual: Operacion "=" Operacion
              | pertenece: Operacion "in" Operacion )
  > left conjuncion: Operacion "and" Operacion
  > left disyuncion: Operacion "or" Operacion
  > left ( flecha: Operacion "-\>" Operacion | flechaDoble: Operacion "=\>" Operacion )
  ;
```

La precedencia y la asociatividad son exactamente las del Proyecto 1. La asociatividad de cada nivel se tomó de la forma de cada regla original. Las reglas de la forma X (op X)*, como ExpresionSuma, ExpresionMult, ExpresionAnd, ExpresionOr y ExpresionFlecha, agrupan hacia la izquierda: a - b - c es (a - b) - c, y por eso son left. ExpresionPotencia se llamaba a sí misma por la derecha, así que 2 **3** 2 es 2 **(3** 2): right. ExpresionComparacion tenía la forma X (op X)?, que permite una sola comparación, así que a < b < c es un error: non-assoc. Los operadores unarios neg y - quedan por encima de la potencia, como en ExpresionUnaria, de modo que -x **2 es (-x)** 2.

Por qué. Es la forma normal de escribir expresiones en Rascal, y es mucho más fácil de leer y de mantener. Además, mejora mucho el AST: con la cascada, el número 5 quedaría envuelto en nueve nodos (uno por nivel) antes de llegar al valor. Con prioridades queda directamente, y 2 + 3 es simplemente suma(2, 3). Por la misma razón desaparece OperadorComparacion: cada operador de comparación es una alternativa con su propio constructor (menor, mayorIgual, pertenece, etc.), en vez de un constructor genérico con el operador como texto. Agrupacion también deja de ser un no terminal aparte y queda como la alternativa agrupacion de ElementoBase. En el documento del Proyecto 1 editado se conserva la cascada en EBNF, que sigue siendo correcta y define la misma precedencia.

Adaptación 4. Listas separadas por comas

Cambio. Las reglas de la forma X ("," X)* se escriben con la notación de Rascal para listas separadas: {X ","}+ (una o más) o {X ","}* (cero o más). Por ejemplo:

```
syntax ConstructorParametros = constructor: "(" {ParametroConstructor ","}+ ")";
syntax Argumentos = argumentos: "(" {Expresion ","}* ")";
syntax Tupla = tuplaExp: "(" Expresion "," {Expresion ","}+ ")";
syntax Parametros = parametros: {Parametro ","}*;
syntax VariableIteracion = varSimple: Id | varTupla: "(" Id "," {Id ","}+ ")";
```

Por qué. Describen exactamente las mismas listas, pero al construir el AST Rascal convierte {X ","}+ directamente en una lista de X y descarta las comas. Con la forma original, el primer elemento y los demás quedaban en partes distintas del árbol. Las tuplas y las variables de iteración siguen exigiendo al menos dos elementos, igual que en el Proyecto 1.

Adaptación 5. Grupos anónimos convertidos en reglas con nombre

Cambio. Los grupos entre paréntesis sin nombre se convirtieron en no terminales propios:

```
syntax DefinicionObjeto = @category="class" objeto: "object" !>> [a-zA-Z0-9] Id
  DefinicionGenerico? ConstructorParametros? Herencia?;
syntax Herencia = herencia: "extends" !>> [a-zA-Z0-9] Id;
syntax DefinicionGenerico = generico: "[" {ParametroGenerico ","}+ "]";
syntax ParametroGenerico = parametroGenerico: Varianza? Id;
syntax Varianza = covariante: "+" | contravariante: "-";
syntax DeclaracionVariable = declaracion: Modificador Id AnotacionTipo? "=" Expresion;
syntax Modificador = inmutable: "val" !>> [a-zA-Z0-9] | mutable: "var" !>> [a-zA-Z0-9];
syntax AnotacionTipo = anotacion: ":" Tipo;
syntax Parametro = parametroMetodo: Id ":" Tipo;
```

Por qué. Rascal construye el AST automáticamente (con la función implode) usando los nombres de las reglas y de sus alternativas. Un grupo anónimo como ("extends" Identificador)? o ("+" | "-")? no tiene nombre, así que no corresponde a ningún constructor del AST, y la construcción falla o queda confusa. Con un nombre propio, cada parte tiene un constructor claro: herencia("Animal"), covariante(), inmutable(), etc. Los programas aceptados son exactamente los mismos.

Adaptación 6. Etiquetas en todas las reglas y relación con el AST

Cambio. Cada alternativa de cada regla tiene una etiqueta, el nombre antes de los dos puntos (programa, objeto, bloque, suma, ifConElse, nueva, sobrescribir, cadena, etc.). Cada etiqueta es única en toda la gramática.

Por qué. Las etiquetas son los nombres de los constructores del AST (archivo AST.rsc). Para cada etiqueta hay un constructor con el mismo nombre y los mismos hijos en el mismo orden, sin contar los símbolos fijos como "object", "(" o ",". Los léxicos se convierten en str (Entero, en int), los opcionales X? y las listas se convierten en list[X]. Si una etiqueta no coincide con su constructor, la construcción del AST falla, así que mantenerlas únicas y consistentes es lo que permite que la gramática y el AST correspondan. Un string completo queda en el AST como texto (litString), incluyendo sus interpolaciones.

Adaptación 7. Resaltado de sintaxis

Cambio. Se agregaron anotaciones @category a algunas reglas (Comment, variable, number, string, class, method) y el archivo Plugin.rsc, que registra el lenguaje en VS Code para los archivos con extensión .olang.

Por qué. Lo pide el enunciado del Proyecto 2. Las anotaciones no cambian qué programas se aceptan; solo le dicen al editor de qué color mostrar cada parte. Plugin.rsc conecta el parser generado a partir de la gramática con el editor, que así marca también los errores de sintaxis.

PARTE 3. RESUMEN DE LO QUE CAMBIÓ EN EL LENGUAJE

Sigue igual que en el Proyecto 1: la estructura de un programa (definiciones seguidas del método main), las definiciones de objetos con genéricos, constructor y herencia, las definiciones de métodos (Objeto.metodo = method(...)), new con cuerpo de override, las tuplas, la interpolación en strings, la precedencia y la asociatividad de los operadores, y que a < b < c es un error.

Casos que cambiaron:

- negative, valores y newton como identificadores. Antes: ambiguos. Ahora: siempre son identificadores.
- val if = 3 (palabra reservada como nombre). Antes: aceptado por la regla, prohibido solo en la nota. Ahora: error de sintaxis.
- if a then b else c + 1. Antes: ambiguo. Ahora: siempre if a then b else (c + 1). Para la otra lectura: (if a then b else c) + 1.
- 1 + if a then b else c. Antes: ambiguo. Ahora: error; se escribe 1 + (if a then b else c).
- if a then if b then c else d. Antes: ambiguo (else colgante). Ahora: el else pertenece al if más cercano.
- if c then val x = 1. Antes: permitido. Ahora: error; se escribe if c then { val x = 1 }.
- { x -1 }. Antes: ambiguo. Ahora: siempre la resta x - 1.
- new Cat(1). Antes: ambiguo. Ahora: una sola lectura.
- 1(2) o (f)(x). Antes: permitidos. Ahora: error.
- String[Integer] como tipo. Antes: permitido. Ahora: error.
- Espacios dentro de un string. Antes: se perdían. Ahora: se conservan.
- Caracteres permitidos en un string. Antes: letras, dígitos, espacio y . , : ! Ahora: cualquier carácter salvo ", $, \ y salto de línea, más los escapes \" \$ \\ \n.
- Comentarios //. Antes: no existían. Ahora: permitidos.

Muchas gracias :D